

MANUAL DE UNITY Y GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para el desarrollo colaborativo de videojuegos en Microsoft Windows con PowerShell 7+, Git Credential Manager, UnityYAMLMerge y GameCI

[Git LFS & Assets Binarios](#)[UnityYAMLMerge 3-Way Driver](#)[Visible Meta Files & GUIDs](#)[Multi-Scene & Prefab Isolation](#)[GameCI & GitHub Actions](#)[Catálogo de Incidentes Críticos](#)

Manual de Unity y GitHub: De Novato a Avanzado en PowerShell (Windows)

Plataforma: Microsoft Windows 10 / Windows 11 / Windows Server

Motor: Unity 2022 LTS / Unity 6 (6000.x)

Herramientas: PowerShell 7+, Git for Windows 2.40+, Git LFS, GitHub CLI (`gh`), GitHub Desktop, Git Credential Manager (GCM), Unity Hub, UnityYAMLMerge

Índice de Contenidos

1. [Parte I: Fundamentos y Preparación del Entorno Unity en Windows](#)
 - 1.1 [Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora](#)
 - 1.2 [Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Winget](#)
 - 1.3 [Configuración Crítica del Editor: Visible Meta Files y Force Text](#)
 - 1.4 [La Regla de Oro de los Archivos `.meta` y los GUIDs](#)
 - 1.5 [El Archivo `.gitignore` Oficial y Optimizado para Unity en Windows](#)
 - 1.6 [Configuración Exhaustiva de Git LFS con `.gitattributes`](#)
2. [Parte II: Flujo de Trabajo Esencial Diario \(Nivel Novato\)](#)
 - 2.1 [Inicializar y Publicar un Proyecto en GitHub \(PowerShell y GitHub Desktop\)](#)
 - 2.2 [Clonación Correcta de Proyectos con Git LFS en Windows](#)
 - 2.3 [El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits](#)
 - 2.4 [Conventional Commits Aplicados al Desarrollo de Videojuegos](#)
 - 2.5 [Sincronización sin Romper la Cache \(`pull --rebase` vs Fetch en Desktop\)](#)
3. [Parte III: Ramas, Fusiones y Estrategias Colaborativas \(Nivel Intermedio\)](#)
 - 3.1 [Estrategia de Ramas en Equipos de Videojuegos \(PowerShell y Desktop\)](#)
 - 3.2 [Arquitectura de Escenas Divididas \(Multi-Scene Editing Aditivo\)](#)
 - 3.3 [Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes](#)
 - 3.4 [Configuración de UnityYAMLMerge en Windows como Mergetool](#)
 - 3.5 [Uso de Git Stash, Historial y Reversión Segura \(PowerShell y Desktop\)](#)
4. [Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente](#)
 - 4.1 [Tema 1: Prevención Arquitectónica de Conflictos en Unity](#)
 - 4.2 [Tema 2: Conflictos en Archivos `.meta` \(GUID Desincronizado\)](#)
 - 4.3 [Tema 3: Conflictos en Scripts C# \(`.cs` \)](#)
 - 4.4 [Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge \(PowerShell y Desktop\)](#)

- 4.5 [Tema 5: Forzar una Versión Completa de Asset \(`--ours` vs `--theirs` \)](#)
 - 4.6 [Tema 6: Cambios Locales en el Editor al Hacer Pull](#)
 - 4.7 [Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS](#)
 - 4.8 [Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock](#)
 - 4.9 [Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano](#)
5. [Parte V: Herramientas Modernas de Productividad Avanzada](#)
- 5.1 [Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Recargar `Library/`](#)
 - 5.2 [Depuración Binaria con Git Bisect y Blame en C#](#)
 - 5.3 [GitHub Codespaces desde PowerShell](#)
 - 5.4 [GitHub Copilot CLI en PowerShell](#)
 - 5.5 [Git Hooks y Validación Pre-commit de Archivos `.meta` en Windows](#)
6. [Parte VI: Gestión de Paquetes UPM y Dependencias](#)
- 6.1 [Instalación de Paquetes Mediante URLs de Git en UPM](#)
 - 6.2 [Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens](#)
7. [Parte VII: Automatización CI/CD con GitHub Actions y GameCI](#)
- 7.1 [Arquitectura de GameCI para StandaloneWindows64](#)
 - 7.2 [Activación de Licencias de Unity en GitHub Actions](#)
 - 7.3 [Pipeline Automatizado: Tests y Compilación StandaloneWindows64 \(.exe\)](#)
 - 7.4 [Subida Automática de Instaladores a GitHub Releases](#)
8. [Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos](#)
- 8.1 [Branch Protection Rules y Rulesets en Windows](#)
 - 8.2 [Gestión de Secretos para APIs de Juegos \(Steam, Photon, Azure\)](#)
 - 8.3 [Gobernanza con `CODEOWNERS` para Artistas y Programadores](#)
9. [Parte IX: Distribución y Despliegue con GitHub Releases](#)
- 9.1 [Creación de Releases con Tags Semánticos desde PowerShell](#)
 - 9.2 [Empaquetado y Publicación de Archivos ZIP e Instaladores de Windows](#)
10. [Parte X: Catálogo Maestro de Incidentes Críticos de Unity en Windows](#)
- 10.1 [Incidente 1: Bloqueo de Archivos por el Proceso de Unity \(`unlink failed` / `Permission Denied` \)](#)
 - 10.2 [Incidente 2: Longitud de Ruta Máxima de Windows Excedida \(`Filename too long` \)](#)
 - 10.3 [Incidente 3: Corrupción de Finales de Línea CRLF vs LF en Scripts C# y YAML](#)
 - 10.4 [Incidente 4: "Missing Script" Masivo por Desincronización de GUIDs](#)
 - 10.5 [Incidente 5: Subida Accidental de `Library/` o Archivos Temporales de Visual Studio](#)
 - 10.6 [Incidente 6: Push Rechazado por Archivo > 100 MB Atrapado en Commits Locales](#)

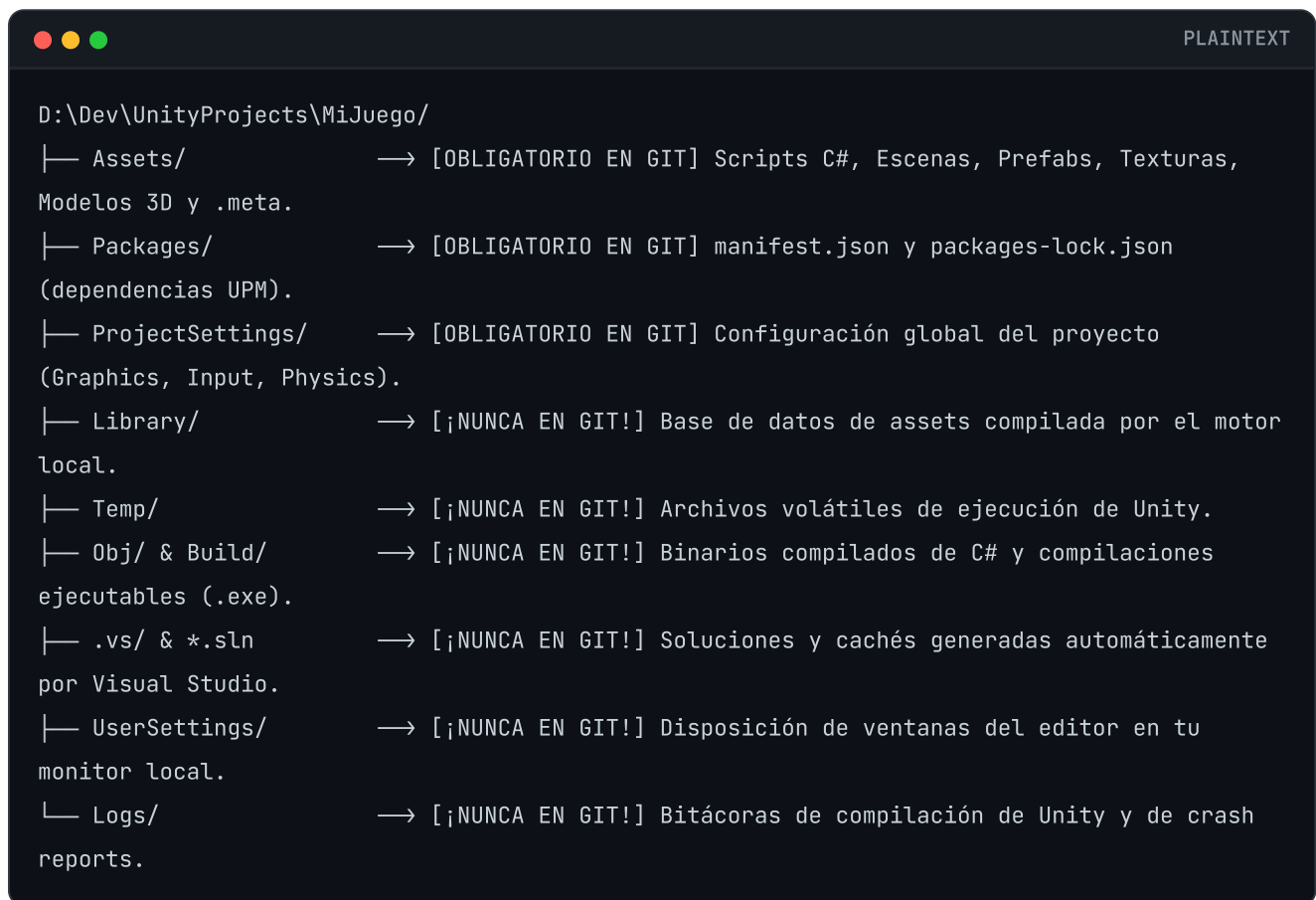
- 10.7 [Incidente 7: Límite de Ancho de Banda de Git LFS Superado](#)
 - 10.8 [Incidente 8: Conflictos de Visual Studio con `.csproj` y `.sln` en Git](#)
 - 10.9 [Incidente 9: Fuga de Claves de Steamworks o Azure en ScriptableObject](#)
 - 10.10 [Incidente 10: Regeneración Segura y Limpia de la Caché Local en Windows](#)
-

Parte I: Fundamentos y Preparación del Entorno

Unity en Windows

1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora

En Windows, los proyectos de Unity generan decenas de miles de archivos temporales al compilar con Visual Studio o Rider:



```
D:\Dev\UnityProjects\MiJuego/
├── Assets/                → [OBLIGATORIO EN GIT] Scripts C#, Escenas, Prefabs, Texturas,
Modelos 3D y .meta.
├── Packages/              → [OBLIGATORIO EN GIT] manifest.json y packages-lock.json
(dependencias UPM).
├── ProjectSettings/       → [OBLIGATORIO EN GIT] Configuración global del proyecto
(Graphics, Input, Physics).
├── Library/               → [¡NUNCA EN GIT!] Base de datos de assets compilada por el motor
local.
├── Temp/                  → [¡NUNCA EN GIT!] Archivos volátiles de ejecución de Unity.
├── Obj/ & Build/          → [¡NUNCA EN GIT!] Binarios compilados de C# y compilaciones
ejecutables (.exe).
├── .vs/ & *.sln           → [¡NUNCA EN GIT!] Soluciones y cachés generadas automáticamente
por Visual Studio.
├── UserSettings/          → [¡NUNCA EN GIT!] Disposición de ventanas del editor en tu
monitor local.
└── Logs/                  → [¡NUNCA EN GIT!] Bitácoras de compilación de Unity y de crash
reports.
```

1.2 Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Winget

Abre **PowerShell 7** (o Windows Terminal) como Administrador y ejecuta:

```
# 1. Instalar Git for Windows, Git LFS, GitHub CLI y GitHub Desktop
winget install --id Git.Git -e --source winget
winget install --id GitHub.cli -e --source winget
winget install --id GitHub.GitHubDesktop -e --source winget

# 2. Inicializar Git LFS globalmente
git lfs install

# 3. Configurar soporte para rutas largas en Windows (>260 caracteres)
git config --system core.longpaths true

# 4. Habilitar Git Credential Manager para Windows
git config --global credential.helper manager
```

¿Qué hace este comando?

Instala el motor completo de Git, GitHub CLI, GitHub Desktop con soporte para el administrador de credenciales de Windows (GCM), activa los filtros globales de Git LFS y desbloquea el límite histórico de 260 caracteres de Windows NTFS (`core.longpaths true`), vital para proyectos complejos de Unity con carpetas profundamente anidadas.

1.3 Configuración Crítica del Editor: Visible Meta Files y Force Text

Antes de agregar el proyecto a Git, verifica estos parámetros en Unity:

1. Abre tu proyecto en Unity en Windows.
2. Navega al menú superior: **Edit** → **Project Settings** → **Version Control**.
 - En el campo **Mode**, selecciona: `Visible Meta Files`.
3. Navega a **Edit** → **Project Settings** → **Editor**.
 - En **Asset Serialization Mode**, selecciona: `Force Text`.

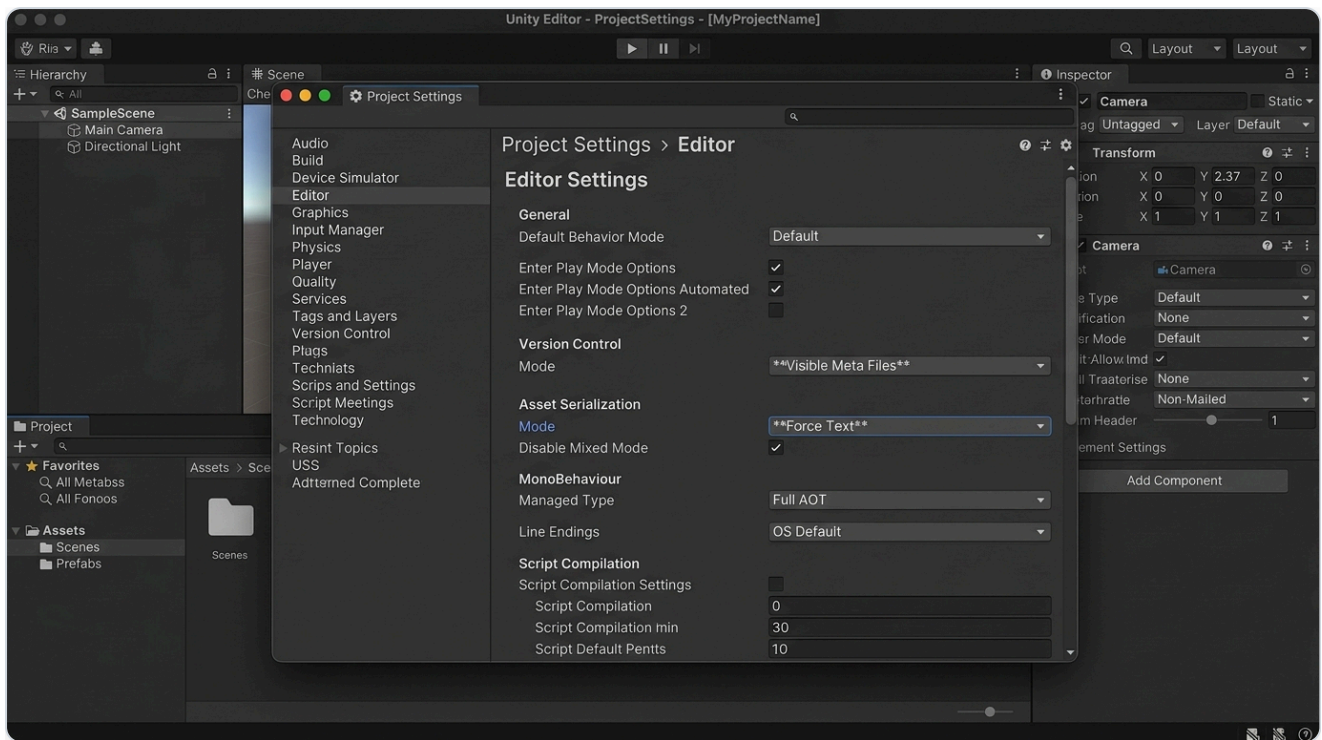


Figura 1.1: Configuración obligatoria en Unity Project Settings (Editor) estableciendo Visible Meta Files y Force Text para serialización YAML en Windows.

IMPORTANT

Force Text obliga al motor a serializar todas las escenas (`.unity`), prefabs (`.prefab`) y materiales (`.mat`) en texto plano **YAML legible** en lugar del formato binario propietario de Unity, permitiendo a Git generar diffs precisos y resolver conflictos.

1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs

Cada asset en `Assets\` cuenta con un archivo gemelo `.meta` que contiene su GUID único:

```
fileFormatVersion: 2
guid: 3f8a92b104c84a56a1b2c3d4e5f60718
```

CAUTION

LA REGLA FUNDAMENTAL:

Si mueves, renombras o eliminas un asset desde el Explorador de Windows o la consola de PowerShell, **debes mover, renombrar o eliminar idénticamente su archivo `.meta`**. Nunca hagas un commit de un asset sin su respectivo `.meta`.

1.5 El Archivo `.gitignore` Oficial y Optimizado para Unity en Windows

Crea el archivo `.gitignore` en la raíz del proyecto:

```
# =====  
# .gitignore para Unity en Microsoft Windows  
# =====  
  
[Ll]ibrary/  
[Tt]emp/  
[Oo]bj/  
[Bb]uild/  
[Bb]uilds/  
[Ll]ogs/  
[Uu]ser[Ss]ettings/  
[Mm]emoryCaptures/  
  
# Asset server cache  
sysinfo.txt  
*.stackdump  
  
# Visual Studio / VS Code / JetBrains Rider  
.vs/  
.idea/  
*.csproj  
*.unityproj  
*.sln  
*.suo  
*.user  
*.userprefs  
*.pidb  
*.booproj  
*.svd  
*.pdb  
*.opendb  
*.VC.db  
  
# Compilaciones Windows (.exe / .dll)  
*.exe  
*.dll  
*.apk  
*.unitypackage  
  
# Archivos del Explorador de Windows  
Thumbs.db
```

```
ehthumbs.db  
[Dd]esktop.ini  
$RECYCLE.BIN/
```

1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`

Guarda este archivo `.gitattributes` en la raíz del repositorio:

```
# =====  
# Configuración Git LFS para Unity en Windows  
# =====  
  
# Formatos 3D y Rigs  
*.fbx filter=lfs diff=lfs merge=lfs -text  
*.obj filter=lfs diff=lfs merge=lfs -text  
*.blend filter=lfs diff=lfs merge=lfs -text  
*.dae filter=lfs diff=lfs merge=lfs -text  
*.max filter=lfs diff=lfs merge=lfs -text  
  
# Texturas e Imágenes Pesadas  
*.psd filter=lfs diff=lfs merge=lfs -text  
*.tga filter=lfs diff=lfs merge=lfs -text  
*.png filter=lfs diff=lfs merge=lfs -text  
*.jpg filter=lfs diff=lfs merge=lfs -text  
*.exr filter=lfs diff=lfs merge=lfs -text  
*.hdr filter=lfs diff=lfs merge=lfs -text  
*.tif filter=lfs diff=lfs merge=lfs -text  
  
# Audio y Vídeo  
*.wav filter=lfs diff=lfs merge=lfs -text  
*.mp3 filter=lfs diff=lfs merge=lfs -text  
*.ogg filter=lfs diff=lfs merge=lfs -text  
*.mp4 filter=lfs diff=lfs merge=lfs -text  
*.mov filter=lfs diff=lfs merge=lfs -text  
  
# Archivos Comprimidos  
*.zip filter=lfs diff=lfs merge=lfs -text  
*.7z filter=lfs diff=lfs merge=lfs -text  
  
# Bloqueo Concurrente para Binarios  
*.fbx lockable  
*.psd lockable  
*.blend lockable  
*.wav lockable  
  
# Normalización de Saltos de Línea y Merge Driver de Unity  
*.cs text diff=csharp eol=lf  
*.json text eol=lf  
*.shader text eol=lf
```

```
*.unity merge=unityyamlmerge eol=lf
*.prefab merge=unityyamlmerge eol=lf
*.mat merge=unityyamlmerge eol=lf
*.asset merge=unityyamlmerge eol=lf
*.meta text eol=lf
```

Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)

2.1 Inicializar y Publicar un Proyecto en GitHub (PowerShell y GitHub Desktop)

Modalidad A: Vía PowerShell 7



```
Set-Location "D:\Dev\UnityProjects\MiJuego"
git init
git lfs install
git add .gitattributes .gitignore
git commit -m "chore: inicializar configuracion Unity con LFS y gitignore"
git branch -M main
gh repo create MiJuegoWindows --private --source=. --remote=origin --push
```

¿Qué hace este comando?

Navega a la carpeta del proyecto, inicializa Git y Git LFS, confirma los archivos de configuración base y usa `gh` para crear el repositorio privado en tu cuenta de GitHub y subir la rama `main`.

Modalidad B: Vía GitHub Desktop

1. Abre **GitHub Desktop**.
2. Pulsa en **File** → **Add Local Repository** (o `Ctrl + O`).
3. Haz clic en **Choose...** y selecciona la carpeta de tu proyecto Unity.
4. Si la carpeta aún no tiene Git inicializado, pulsa en el enlace azul **"create a repository"**.
5. Haz clic en el botón azul **Add Repository**.
6. En la barra superior, haz clic en **Publish repository** para subirlo a tu cuenta de GitHub de forma privada o pública.

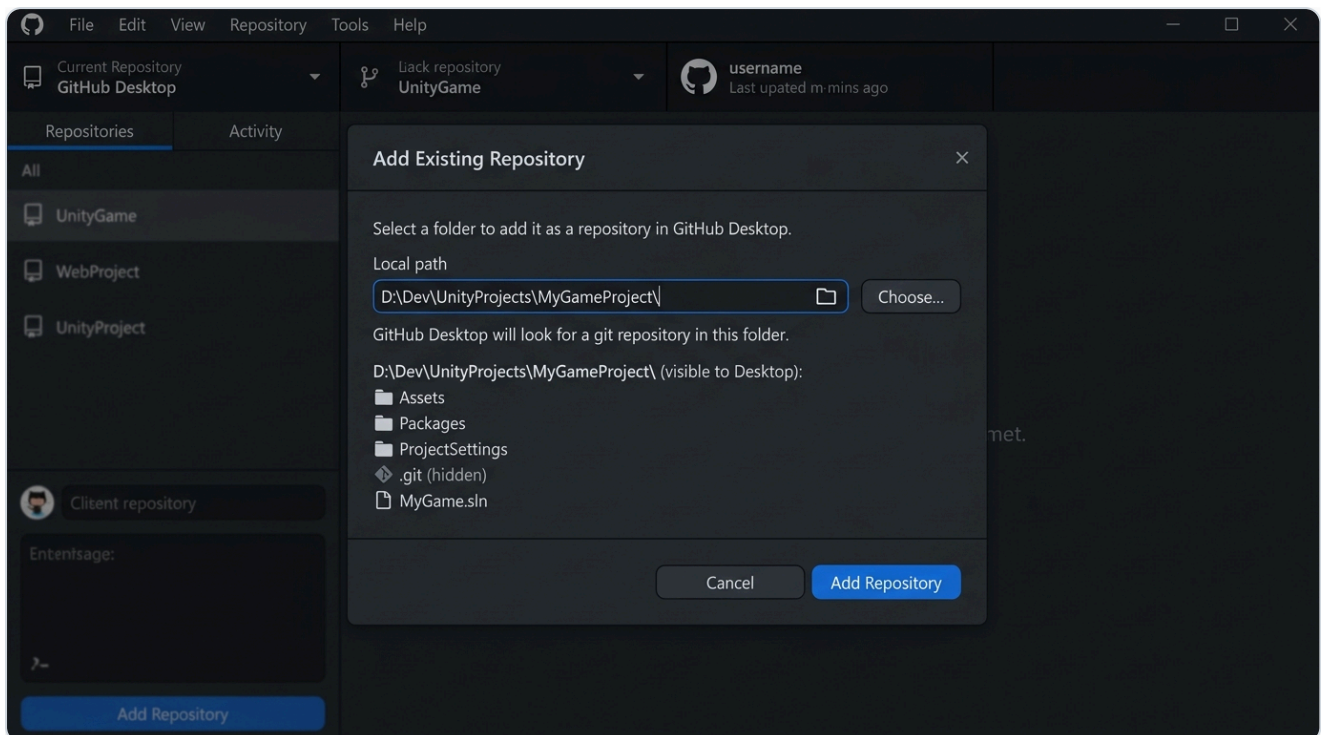


Figura 2.1: Cuadro de diálogo 'Add Existing Repository' en GitHub Desktop vinculando la carpeta del proyecto Unity en Windows.

2.2 Clonación Correcta de Proyectos con Git LFS en Windows

Modalidad A: Vía PowerShell

```
git lfs install
git clone git@github.com:mi-organizacion/videojuego.git
Set-Location videojuego
git lfs pull
```

Modalidad B: Vía GitHub Desktop

1. Ve a **File** → **Clone Repository** (o `Ctrl + Shift + 0`).
2. Selecciona el repositorio de la lista de tu cuenta o introduce la URL.
3. Elige la carpeta de destino local y pulsa **Clone**.
4. GitHub Desktop descargará automáticamente todos los modelos 3D y texturas de LFS en segundo plano.

2.3 El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits

Modalidad A: Vía PowerShell



```
POWERSHELL

# 1. Comprobar archivos modificados
git status

# 2. Agregar scripts y prefabs junto con sus respectivos .meta
git add Assets/Scripts/PlayerController.cs Assets/Scripts/PlayerController.cs.meta
git add Assets/Prefabs/Player.prefab Assets/Prefabs/Player.prefab.meta

# 3. Confirmar commit
git commit -m "feat(player): agregar control de movimiento y fisica con Rigidbody2D"
```

Modalidad B: Vía GitHub Desktop

1. En la pestaña **Changes** de la izquierda, revisa la lista de archivos modificados.
2. Comprueba que cada script `.cs` o prefab tenga marcada su casilla junto a su archivo gemelo `.meta`.
3. Haz clic sobre cualquier archivo para ver en el panel derecho las diferencias exactas de código (*diff*).
4. Rellena el campo **Summary** con el mensaje del commit y opcionalmente añade detalles en **Description**.
5. Pulsa en el botón azul **Commit to main** (o a la rama de trabajo activa).

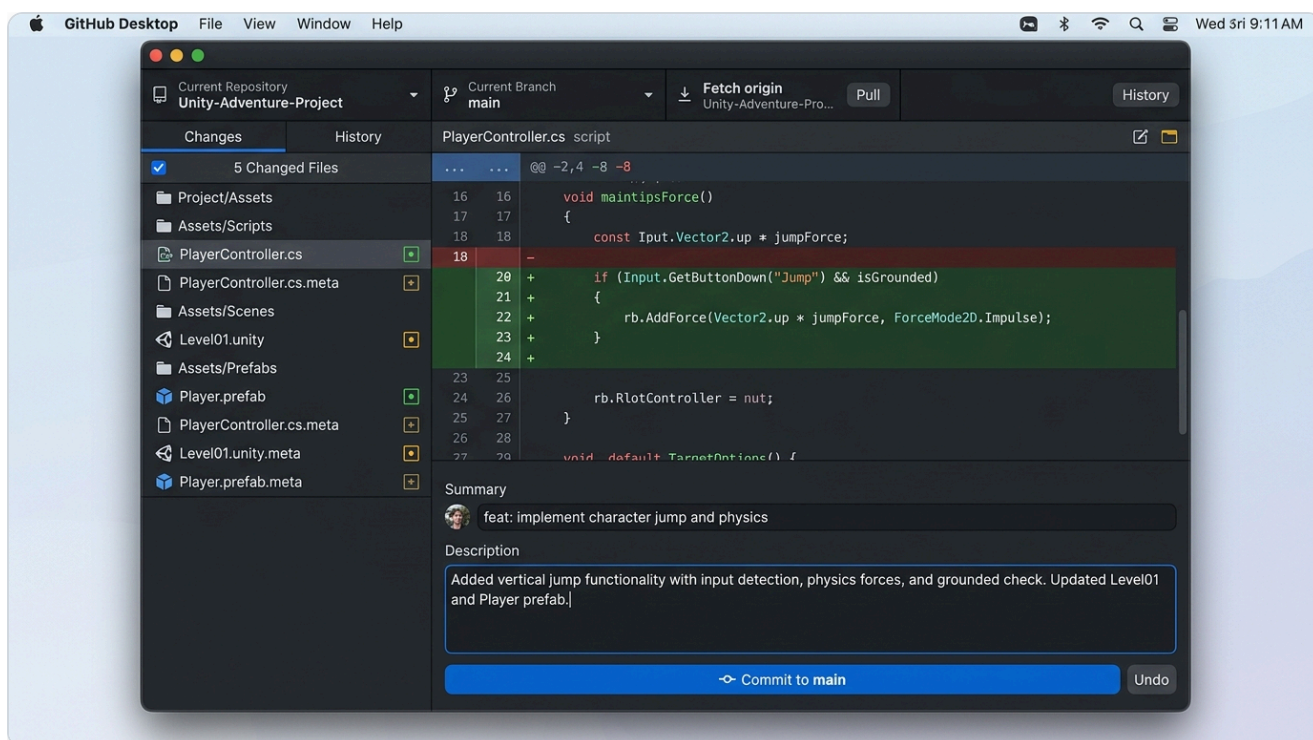


Figura 2.2: Pestaña de cambios en GitHub Desktop verificando la integridad en pareja de archivos y metadatos de Unity antes de confirmar.

2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos

Prefijo	Área de Aplicación en Videojuegos	Ejemplo
feat:	Nuevas mecánicas o sistemas jugables	feat(inventory): crear sistema de slots y arrastre de items
fix:	Corrección de fallos o glitches de físicas	fix(camera): corregir vibracion de Cinemachine al seguir al jugador
art:	Modelos 3D, texturas, animaciones y rigs	art(weapons): anadir modelo 3D y textura PBR de la escopeta
level:	Modificaciones de escenas y diseño de niveles	level(arena): distribuir spawners de oleadas y luces nocturnas
audio:	Efectos de sonido y mezclas de AudioMixer	audio(ambient): integrar pistas de viento y pasos sobre nieve
perf:	Optimizaciones de draw calls y memoria	perf(occlusion): hornear matrices de Occlusion Culling en nivel 1
chore:	Mantenimiento de paquetes UPM o gitignore	chore(upm): actualizar paquete Input System a version 1.7.0

2.5 Sincronización sin Romper la Cache (pull --rebase vs Fetch en Desktop)

Modalidad A: Vía PowerShell

```
git pull --rebase origin main
git push origin main
```

El uso de `--rebase` evita los merge commits artificiales que alteran la fecha de modificación de los archivos y obligan al motor a reimportar la base de datos de assets.

Modalidad B: Vía GitHub Desktop

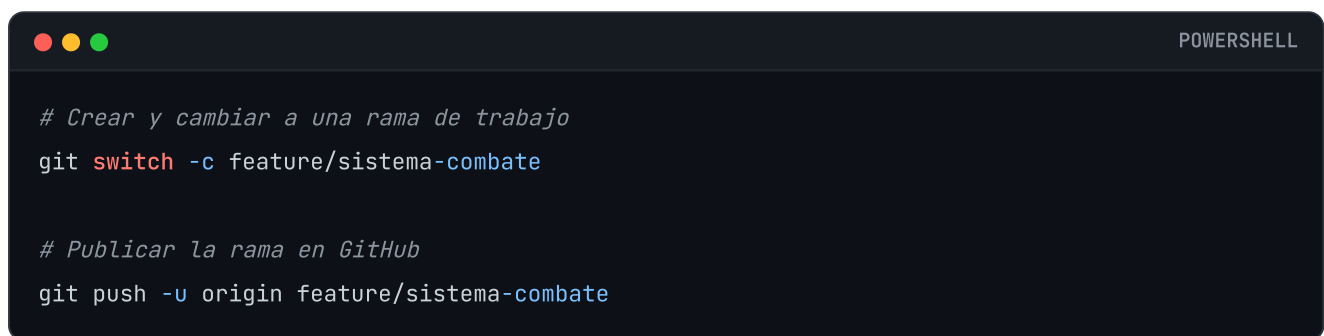
1. Pulsa en **Fetch origin** en la barra superior para buscar actualizaciones remotas.
2. Si hay commits nuevos de tus compañeros, pulsa en el botón **Pull origin**.

3. Envía tus cambios pendientes haciendo clic en **Push origin**.

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Estrategia de Ramas en Equipos de Videojuegos (PowerShell y Desktop)

Modalidad A: Vía PowerShell



```
# Crear y cambiar a una rama de trabajo
git switch -c feature/sistema-combate

# Publicar la rama en GitHub
git push -u origin feature/sistema-combate
```

Modalidad B: Vía GitHub Desktop

1. En la barra superior, haz clic en el menú **Current Branch**.
2. Escribe el nombre de tu rama (ej. `feature/sistema-combate`).
3. Haz clic en el botón azul **New branch....**
4. Haz clic en **Publish branch** para sincronizarla con GitHub.

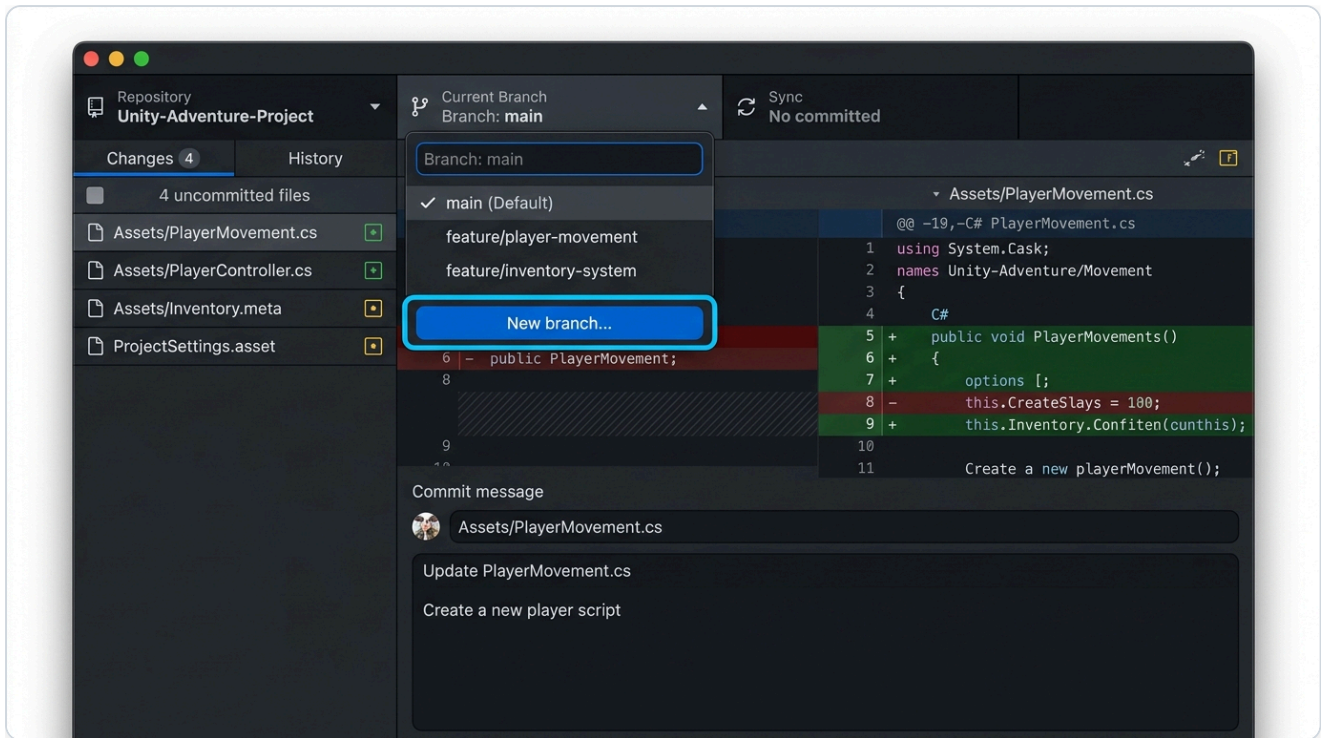


Figura 3.1: Menú desplegable 'Current Branch' en GitHub Desktop facilitando la creación y conmutación de ramas aisladas en Windows.

3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)

En videojuegos, editar la misma escena al mismo tiempo genera colisiones catastróficas. La solución estándar en la industria es dividir el mundo en múltiples escenas aditivas:

```

Nivel_Bosque/
├─ Nivel_Core.unity      → Cámaras, GameManagers, UI Canvas.
├─ Nivel_Arte.unity     → Terreno, vegetación, rocas y props (Artistas).
├─ Nivel_Luces.unity    → Post-processing, luces y sondas (Iluminadores).
└─ Nivel_Gameplay.unity → Triggers, enemigos y zonas de muerte (Diseñadores).
  
```

Script C# para Carga Aditiva en Tiempo de Ejecución:

Guarda en `Assets\Scripts\AdditiveLoader.cs` :

```

using UnityEngine;
using UnityEngine.SceneManagement;

public class AdditiveLoader : MonoBehaviour
{
    [SerializeField] private string[] subScenes = { "Nivel_Arte", "Nivel_Luces",
    "Nivel_Gameplay" };

    private void Start()
    {
        foreach (string scene in subScenes)
        {
            if (!SceneManager.GetSceneByName(scene).isLoaded)
            {
                SceneManager.LoadSceneAsync(scene, LoadSceneMode.Additive);
            }
        }
    }
}

```

3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes

- Edita componentes siempre en el **Prefab Mode**, no directamente en la escena.
- Emplea **Prefab Variants** para variantes de enemigos o props. Las variantes solo guardan las diferencias (*overrides*), evitando conflictos de fusión en el archivo base.

3.4 Configuración de UnityYAMLMerge en Windows como Mergetool

Configura el motor oficial de fusión tridireccional de Unity en PowerShell:

```
# Obtener la ruta del Unity Editor instalado más reciente en Program Files
$UnityPath = Get-ChildItem "C:\Program
Files\Unity\Hub\Editor\*\Editor\Data\Tools\UnityYAMLMerge.exe" | Select-Object -Last 1 -
ExpandProperty FullName

# Configurar el driver de fusión en Git
git config --global merge.unityyamlmerge.name "Unity Smart Merge"
git config --global merge.unityyamlmerge.driver "\"$UnityPath\" merge -h -p -- \"%0\" \"%B\"
\"%A\" \"%A\""
git config --global merge.unityyamlmerge.trustExitCode true
git config --global merge.unityyamlmerge.recursive binary
```

¿Qué hace este comando?

Localiza la versión instalada de Unity y conecta `UnityYAMLMerge.exe` con Git. Cuando ocurra un conflicto en un archivo `.unity` o `.prefab`, UnityYAMLMerge fusionará automáticamente los GameObjects modificados.

3.5 Uso de Git Stash, Historial y Reversión Segura (PowerShell y Desktop)

Modalidad A: Vía PowerShell

```
# Guardar cambios sin confirmar
git stash save "WIP: ajuste de inventario"

# Cambiar de rama y revisar bug
git switch main

# Restaurar cambios
git switch feature/mi-rama
git stash pop
```

Modalidad B: Vía GitHub Desktop

1. Al cambiar de rama con cambios sucios, pulsa en **"Leave my changes on [rama-actual]"** (crea un stash).
2. Para restaurar, pulsa en **Restore** dentro de **Stashed Changes** en la barra lateral.

3. Para deshacer un commit con errores: ve a la pestaña **History**, clic derecho sobre el commit → **Revert changes in commit**.

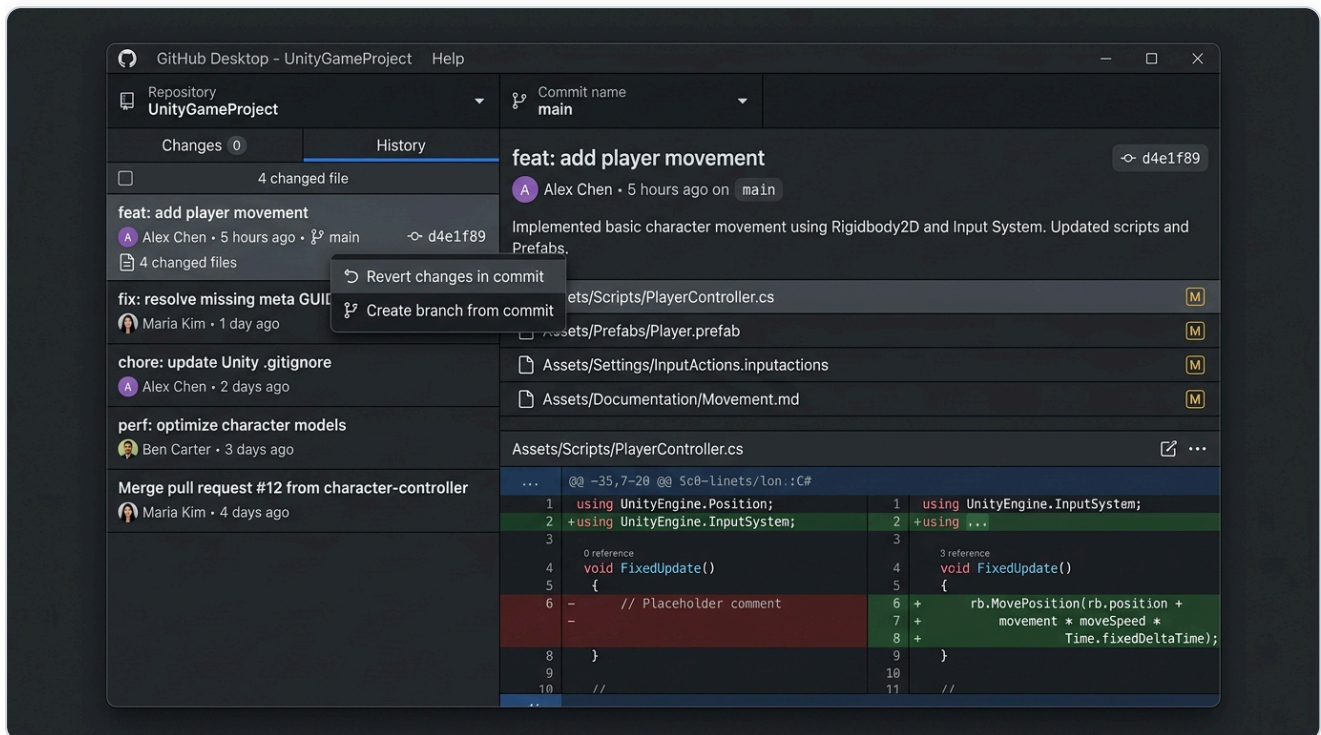


Figura 3.2: Pestaña 'History' en GitHub Desktop permitiendo inspección de diffs y reversión limpia de commits problemáticos.

Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente

4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity

- Organiza las carpetas por subsistemas funcionales.
 - No permitas que dos personas editen la misma escena sin una división aditiva previa.
 - Utiliza prefabs independientes para cada entidad interactiva.
-

4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)



```
# Ver archivos en conflicto
git status
```

1. Abre el archivo `.meta` en VS Code.
2. Si el archivo ya estaba referenciado en escenas por otro desarrollador, conserva el GUID del servidor remoto.
3. Elimina los marcadores de conflicto dejando una sola línea `guid: .`
4. Confirma la resolución:



```
git add Assets\Textures\Heroe.png.meta
git commit -m "fix(meta): resolver colision de GUID en textura de Heroe"
```

4.3 Tema 3: Conflictos en Scripts C# (`.cs`)

Combina las líneas en Visual Studio o Rider, guarda y ejecuta:

```
git add Assets\Scripts\EnemyAI.cs  
git commit -m "fix(ai): fusionar logica de persecucion y ataque a distancia"
```

4.4 Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge (PowerShell y Desktop)

Modalidad A: Vía PowerShell

```
git mergetool -t unityyamlmerge
```

UnityYAMLMerge resolverá de forma semántica los IDs internos de la escena sin corromper el árbol de objetos.

Modalidad B: Vía GitHub Desktop

1. Tras un merge con colisión, aparece la ventana **Resolve conflicts before merging**.
2. En la fila del archivo de escena (`.unity`), haz clic en **Open in UnityYAMLMerge**.
3. Tras la fusión automática, haz clic en el botón azul **Resolve Conflicts** para sellar el merge.

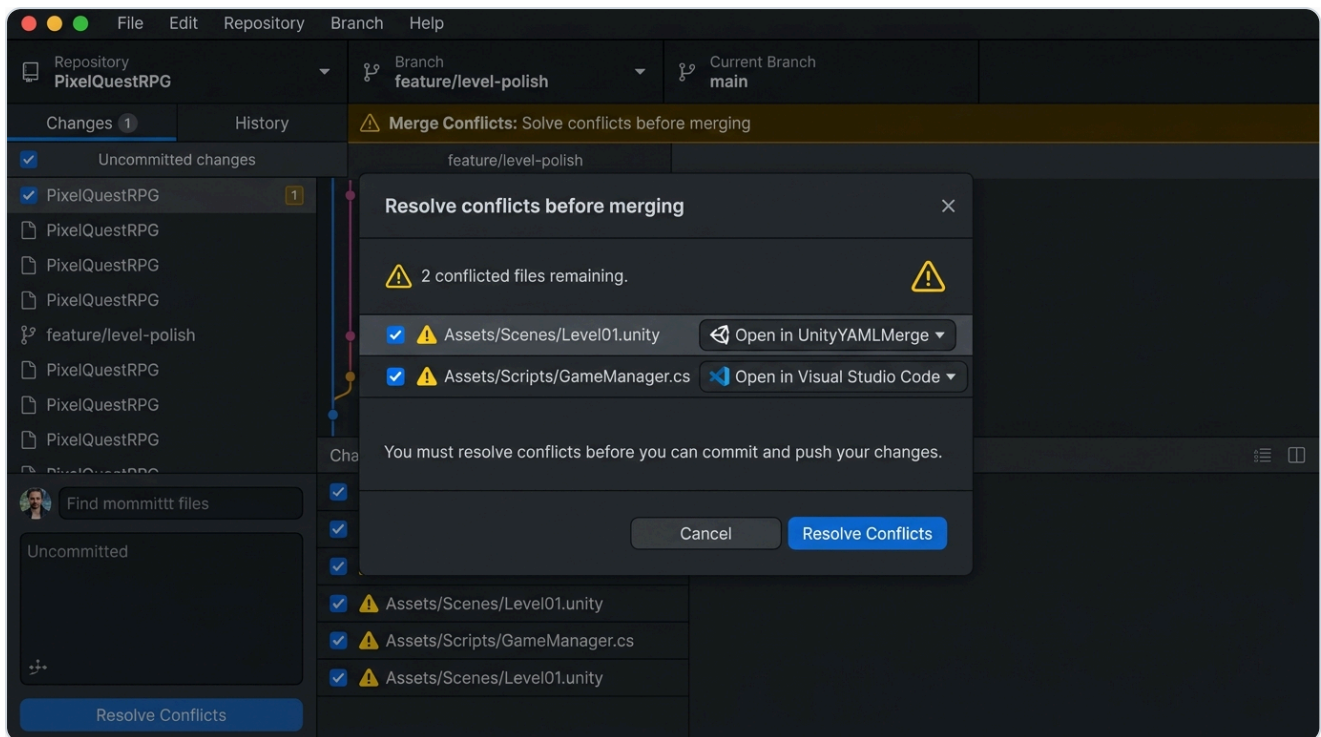


Figura 4.1: Modal de resolución de conflictos en GitHub Desktop en Windows con opciones de apertura en UnityYAMLMerge.

4.5 Tema 5: Forzar una Versión Completa de Asset (--ours vs --theirs)

Modalidad A: Vía PowerShell

```
POWERSHELL

# Conservar tu versión local
git checkout --ours Assets\Scenes\Nivel01.unity
git add Assets\Scenes\Nivel01.unity
git commit -m "resolve: conservar version local de Nivel01"

# O aceptar completamente la versión remota
git checkout --theirs Assets\Scenes\Nivel01.unity
git add Assets\Scenes\Nivel01.unity
git commit -m "resolve: aceptar version del servidor de Nivel01"
```

Modalidad B: Vía GitHub Desktop

En la ventana modal de conflictos, haz clic en la flecha derecha del archivo y selecciona **Use Modified Version** (tuya) o **Use Existing Version** (remota).

4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull

```
git stash save "cambios-locales"
git pull --rebase origin main
git stash pop
```

4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS

```
git fetch origin
git rebase origin/main
git push origin main
```

4.8 Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock

Para evitar que dos artistas trabajen sobre el mismo archivo `.psd` o `.blend` en Windows:

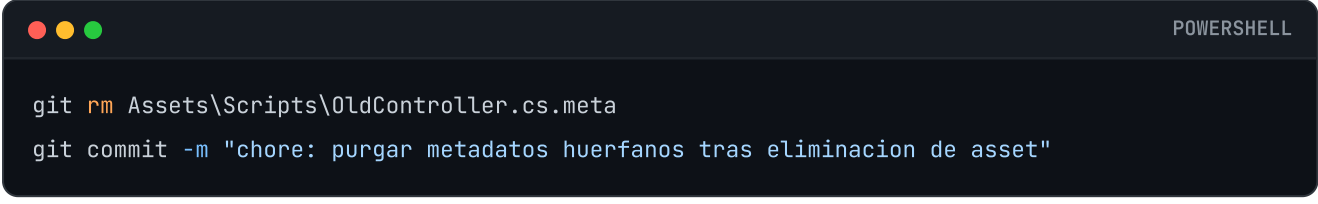
```
# 1. Bloquear archivo antes de editar
git lfs lock Assets\Art\Modelos\JefeFinal.blend

# 2. Consultar bloqueos activos
git lfs locks

# 3. Editar, commitear y subir
git add Assets\Art\Modelos\JefeFinal.blend
git commit -m "art(boss): esculpir detalles de armadura en modelo Blender"
git push origin main

# 4. Desbloquear para el equipo
git lfs unlock Assets\Art\Modelos\JefeFinal.blend
```

4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano



A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner reads "POWERSHELL". The terminal contains two lines of text: a git command to remove a file and a commit message.

```
git rm Assets\Scripts\OldController.cs.meta  
git commit -m "chore: purgar metadatos huérfanos tras eliminacion de asset"
```

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Recargar Library/

```
POWERSHELL

# Crear worktree paralelo en otra carpeta
git worktree add ..\MiJuego-Hotfix hotfix/correccion-audio

# Trabajar en la carpeta secundaria sin tocar el Unity principal
Set-Location ..\MiJuego-Hotfix
git push origin hotfix/correccion-audio

# Al terminar, eliminar el worktree
Set-Location ..\MiJuego
git worktree remove ..\MiJuego-Hotfix
```

5.2 Depuración Binaria con Git Bisect y Blame en C#

```
POWERSHELL

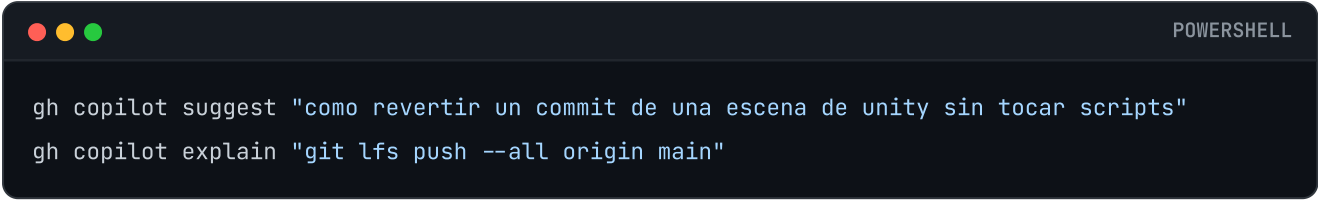
git bisect start
git bisect bad
git bisect good v1.2.0
# Probar y marcar: git bisect good / git bisect bad
git bisect reset
```

5.3 GitHub Codespaces desde PowerShell

```
POWERSHELL

gh codespace create --repo mi-organizacion/MiJuegoWindows --branch main
```

5.4 GitHub Copilot CLI en PowerShell



```
gh copilot suggest "como revertir un commit de una escena de unity sin tocar scripts"
gh copilot explain "git lfs push --all origin main"
```

5.5 Git Hooks y Validación Pre-commit de Archivos .meta en Windows

Crea el archivo `.git\hooks\pre-commit` :



```
#!/bin/sh
MISSING=0
for f in $(git diff --cached --name-only --diff-filter=A | grep '^Assets/'); do
    if [ "${f##*.}" != "meta" ]; then
        if [ ! -f "${f}.meta" ] && ! git diff --cached --name-only | grep -q "^${f}.meta$";
    then
        echo "❌ ERROR: Falta el archivo .meta para: $f"
        MISSING=1
    fi
fi
done
[ $MISSING -eq 1 ] && exit 1 || exit 0
```

Parte VI: Gestión de Paquetes UPM y Dependencias

6.1 Instalación de Paquetes Mediante URLs de Git en UPM

Añade a `Packages\manifest.json` :

```
{
  "dependencies": {
    "com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?
path=src/UniTask/Assets/Plugins/UniTask",
    "com.nevecc.unirx": "https://github.com/nevecc/UniRx.git?path=Assets/Plugins/UniRx"
  }
}
```

6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens

Configura en `C:\Users\<TuUsuario>\.upmconfig.toml` :

```
[npmAuth."https://npm.pkg.github.com/mi-estudio"]
token = "ghp_TU_TOKEN_PERSONAL_CON_SCOPE_READ_PACKAGES"
email = "desarrollador@mi-estudio.com"
alwaysAuth = true
```

Parte VII: Automatización CI/CD con GitHub Actions y GameCI

7.1 Arquitectura de GameCI para StandaloneWindows64

Compila automáticamente binarios de Windows a 64 bits (`.exe`) en runners de GitHub Actions mediante contenedores Docker con Windows target.

7.2 Activación de Licencias de Unity en GitHub Actions

En **Settings** → **Secrets and variables** → **Actions**, define:

- `UNITY_EMAIL`
 - `UNITY_PASSWORD`
 - `UNITY_LICENSE`
-

7.3 Pipeline Automatizado: Tests y Compilación StandaloneWindows64 (.exe)

Crea `.github\workflows\build-windows.yml` :

```

name: Unity Windows Build

on:
  push:
    branches: [ main ]

jobs:
  build:
    name: 🎮 Compilar StandaloneWindows64
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v3
        with:
          path: Library
          key: Library-Windows-${{ hashFiles('Assets/**', 'Packages/**') }}

      - uses: game-ci/unity-builder@v4
        env:
          UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
          UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
          UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
        with:
          targetPlatform: StandaloneWindows64
          buildName: MiJuegoWindows

      - uses: actions/upload-artifact@v4
        with:
          name: Build-Windows-x64
          path: build/StandaloneWindows64

```

7.4 Subida Automática de Instaladores a GitHub Releases

El pipeline genera un archivo comprimido descargable con el ejecutable `.exe` y los datos asociados `_Data` de Unity.

Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos

8.1 Branch Protection Rules y Rulesets en Windows

Protege la rama `main` en GitHub:

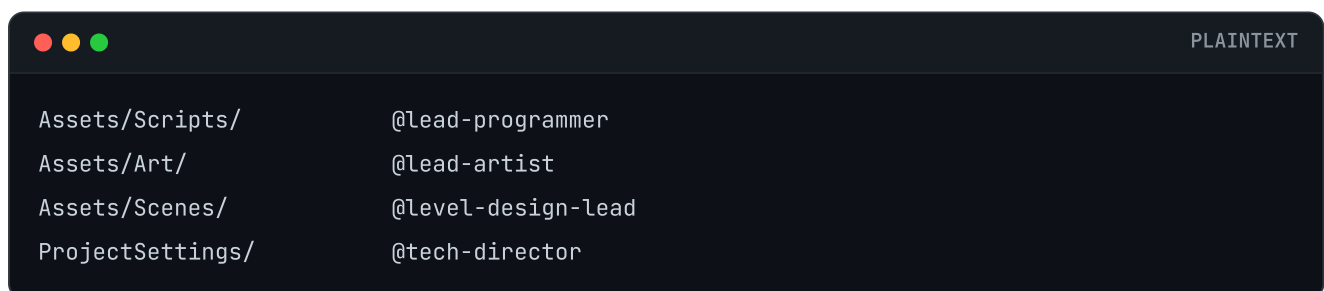
1. Obliga a pasar los tests de GameCI antes de fusionar.
2. Bloquea pushes directos forzando el uso de Pull Requests.

8.2 Gestión de Secretos para APIs de Juegos (Steam, Photon, Azure)

Guarda las credenciales en variables de entorno o archivos `.env` ignorados por Git. Nunca commitees claves de Steamworks o Azure PlayFab en scripts C#.

8.3 Gobernanza con CODEOWNERS para Artistas y Programadores

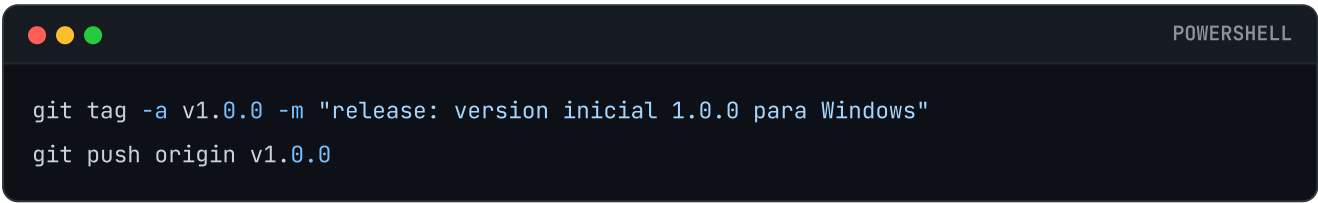
Crea `.github\CODEOWNERS` :



```
Assets/Scripts/      @lead-programmer
Assets/Art/           @lead-artist
Assets/Scenes/        @level-design-lead
ProjectSettings/      @tech-director
```

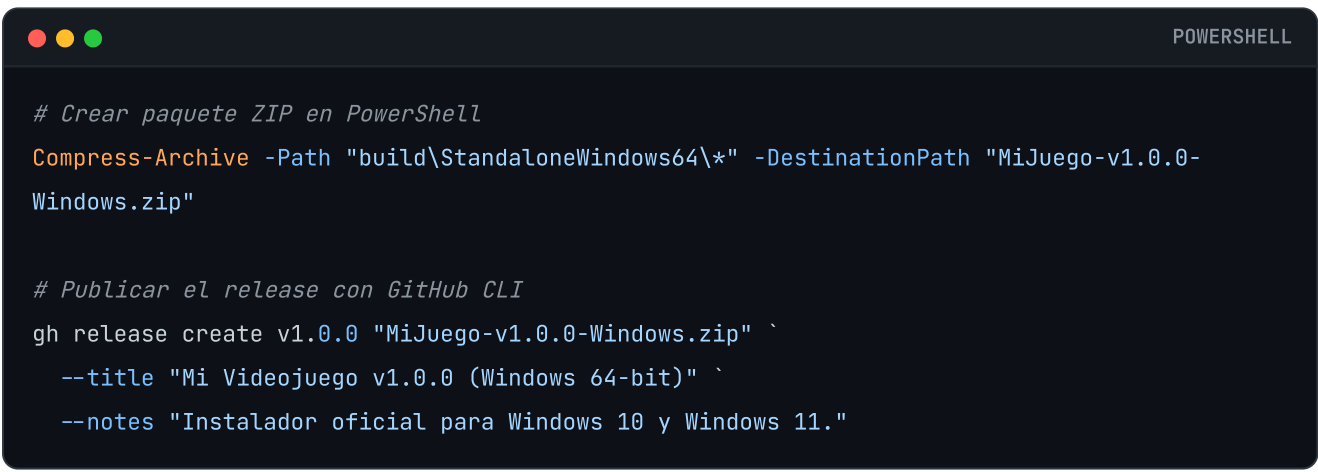
Parte IX: Distribución y Despliegue con GitHub Releases

9.1 Creación de Releases con Tags Semánticos desde PowerShell



```
git tag -a v1.0.0 -m "release: version inicial 1.0.0 para Windows"
git push origin v1.0.0
```

9.2 Empaquetado y Publicación de Archivos ZIP e Instaladores de Windows



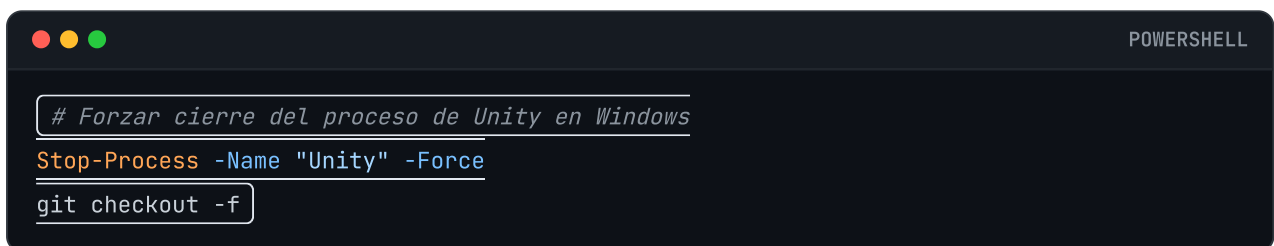
```
# Crear paquete ZIP en PowerShell
Compress-Archive -Path "build\StandaloneWindows64\*" -DestinationPath "MiJuego-v1.0.0-Windows.zip"

# Publicar el release con GitHub CLI
gh release create v1.0.0 "MiJuego-v1.0.0-Windows.zip" `
  --title "Mi Videojuego v1.0.0 (Windows 64-bit)" `
  --notes "Instalador oficial para Windows 10 y Windows 11."
```

Parte X: Catálogo Maestro de Incidentes Críticos de Unity en Windows

10.1 Incidente 1: Bloqueo de Archivos por el Proceso de Unity (`unlink failed / Permission Denied`)

- **Causa:** El Editor de Unity o el demonio AssetDatabase mantiene descriptores de archivo abiertos en Windows, impidiendo a Git conmutar de rama.
- **Solución:**



```
# Forzar cierre del proceso de Unity en Windows
Stop-Process -Name "Unity" -Force
git checkout -f
```

10.2 Incidente 2: Longitud de Ruta Máxima de Windows Excedida (`Filename too long`)

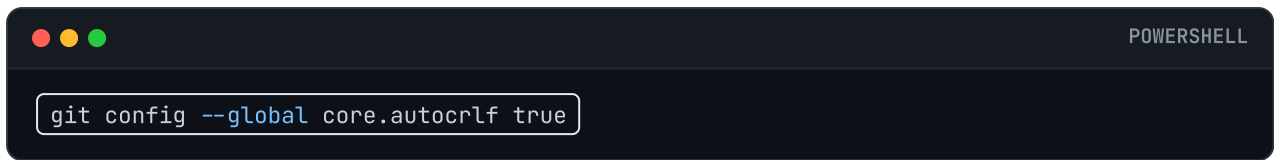
- **Causa:** Rutas que superan los 260 caracteres en `Assets\Packages\com.unity...`.
- **Solución:**



```
git config --system core.longpaths true
```

10.3 Incidente 3: Corrupción de Finales de Línea CRLF vs LF en Scripts C# y YAML

- **Causa:** Windows escribe `\r\n` mientras Unity prefiere `\n` (LF) para diffs legibles.
- **Solución:**



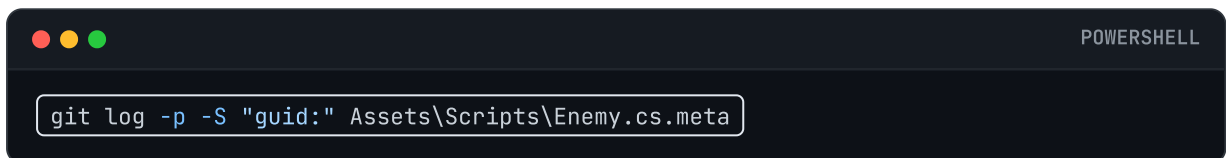
```
git config --global core.autocrlf true
```

Asegúrate de que `.gitattributes` contenga las reglas `eoL=Lf` definidas en la Parte I.

10.4 Incidente 4: "Missing Script" Masivo por Desincronización de GUIDs

- Solución:

1. Busca el GUID previo en el historial:

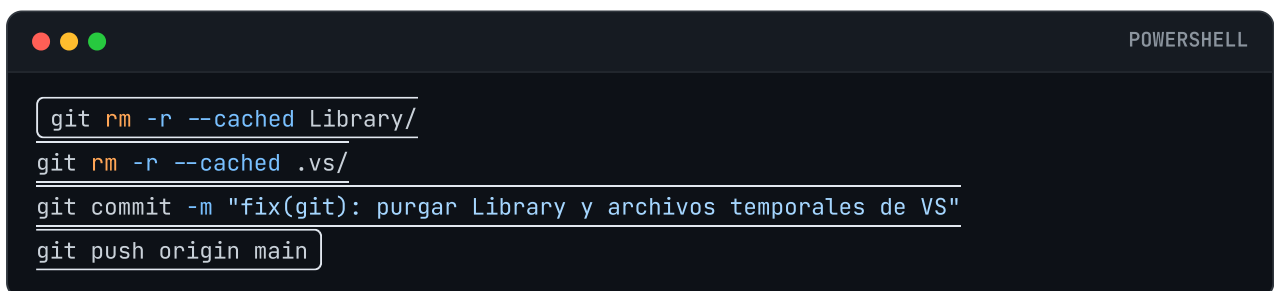


```
git log -p -S "guid:" Assets\Scripts\Enemy.cs.meta
```

2. Restaura el GUID original en el archivo `.meta` y reinicia Unity.

10.5 Incidente 5: Subida Accidental de `Library/` o Archivos Temporales de Visual Studio

- Solución:



```
git rm -r --cached Library/  
git rm -r --cached .vs/  
git commit -m "fix(git): purgar Library y archivos temporales de VS"  
git push origin main
```

10.6 Incidente 6: Push Rechazado por Archivo > 100 MB Atrapado en Commits Locales

- Solución con `git-filter-repo` :

```
POWERSHELL

pip install git-filter-repo
git lfs migrate import --include="*.fbx,*.psd,*.blend,*.wav" --everything
git push origin --force --all
```

10.7 Incidente 7: Límite de Ancho de Banda de Git LFS Superado

- Solución:

```
POWERSHELL

git lfs prune --dry-run
git lfs prune
```

10.8 Incidente 8: Conflictos de Visual Studio con .csproj y .sln en Git

- Solución: Los archivos `.sln` y `.csproj` se autogeneran por Unity. Elimínalos del repositorio y añádelos a `.gitignore`:

```
POWERSHELL

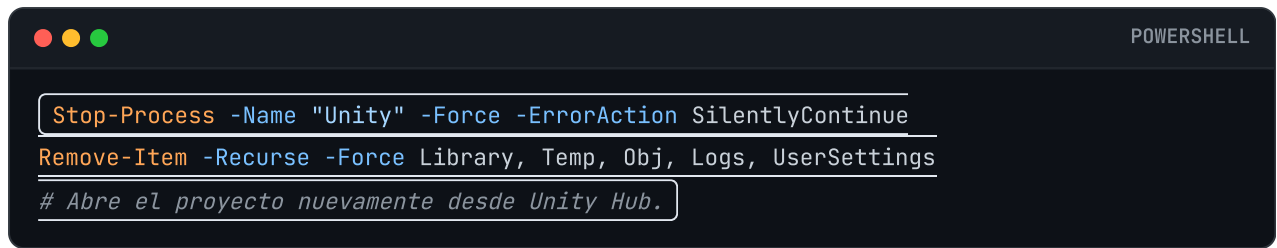
git rm --cached *.sln *.csproj
git commit -m "chore: remover soluciones autogeneradas de Visual Studio"
```

10.9 Incidente 9: Fuga de Claves de Steamworks o Azure en ScriptableObjects

- Solución: Revoca el token en el portal del desarrollador y purga el archivo con `git filter-repo`.

10.10 Incidente 10: Regeneración Segura y Limpia de la Caché Local en Windows

- Solución (El Reset Seguro):



A screenshot of a Windows PowerShell terminal window. The window has a dark background and a title bar with three colored buttons (red, yellow, green) on the left and the word "POWERSHELL" on the right. The terminal contains three lines of code: `Stop-Process -Name "Unity" -Force -ErrorAction SilentlyContinue`, `Remove-Item -Recurse -Force Library, Temp, Obj, Logs, UserSettings`, and a comment line `# Abre el proyecto nuevamente desde Unity Hub.`. The code is color-coded: `Stop-Process` is orange, `-Name` is blue, `"Unity"` is white, `-Force` is blue, `-ErrorAction` is blue, and `SilentlyContinue` is white. The second line has `Remove-Item` in orange, `-Recurse` in blue, `-Force` in blue, and the file names in white. The comment line is in white.

```
Stop-Process -Name "Unity" -Force -ErrorAction SilentlyContinue
Remove-Item -Recurse -Force Library, Temp, Obj, Logs, UserSettings
# Abre el proyecto nuevamente desde Unity Hub.
```